

Session 1.4: Tabular Data Wrangling Part 2: Advanced Filtering, Multi-Segment GroupBy, and Financial Auditing

1:2 DELIVERY RHYTHM

Applied Learning

60 minutes of focused instruction followed by 120 minutes of guided coding and debugging.

SESSION OUTCOME

Operational Competency

Students apply the session concept to a small, verifiable business dataset.

Concepts for This Session

CONCEPT 1

Boolean Masking

Filtering tabular rows using boolean condition vectors combined with bitwise logical operators (& for AND, | for OR, ~ for NOT).

CONCEPT 2

Split-Apply-Combine Pattern

The computational workflow of splitting data into operational groups, applying analytical functions, and combining results into an aggregated table.

CONCEPT 3

Multi-Metric Aggregation (.agg())

Computing different statistical metrics simultaneously across multiple columns (e.g., total expenses, average variance, maximum transaction size).

CONCEPT 4

Relational Merging

Joining two DataFrames based on common business keys (such as Cost_Center_Code), replacing error-prone spreadsheet VLOOKUP and XLOOKUP functions.

How the Concept Works in Practice

FOUNDATION 1

Boolean Indexing at Enterprise Scale

In business reporting, filtering data across multiple conditions requires bitwise operators (&, |) because standard Python and/or operate on single scalar booleans, whereas Pandas evaluates element-wise boolean arrays.

FOUNDATION 2

The Split-Apply-Combine Paradigm

Grouping data by corporate dimensions (e.g., Division_Name, Fiscal_Quarter) to calculate sub-totals and unit economics without manually building pivot tables.

FOUNDATION 3

Relational Joins vs. Fragile Spreadsheet Lookups

Spreadsheet lookup formulas fail when columns are rearranged or when duplicate keys exist. A Pandas merge (how="inner", how="left") explicitly specifies match criteria and logs unmapped rows automatically.

FOUNDATION 4

Automated Audit Exception Flags

Constructing conditional categorical flags (e.g., Variance_Flag when Actual_Expense > Budget_Expense 1.05) to route financial discrepancies directly to corporate controllers.

Build, Test, and Diagnose in the IDE

[1] FILE EXPLORER

Organize Assets

Use the session dataset and keep scripts in a clear workspace.

[2] CODE EDITOR

Write Small Segments

Run each completed segment before expanding the program.

[3] AI AGENT CHAT

Diagnose Errors

Capture the traceback, request an explanation, and review the fix.

[4] INTEGRATED TERMINAL

Verify Execution

Confirm the final program completes with return code 0.

Script Demonstration 1: Tabular Data Wrangling Part 2: Advanced Filtering, Multi-Segment GroupBy, and Financial Auditing

```
import pandas as pd

df = pd.read_csv("data/session_01_04_dataset.csv")
mask = (df["Division_Name"] == "Consumer") &
(df["Actual_Expense_USD"] > df["Budget_Expense_USD"])
overrun_txns = df[mask]
print(f"Consumer Overrun Transactions Count: {len(overrun_txns)}")
```

SYNTAX

Readable Structure

Observe the syntax and indentation that control execution.

RUNTIME

Verified Execution

Run the example in the integrated terminal and inspect the output.

BUSINESS VALUE

Applied Automation

Use the pattern to automate a repeatable business decision.

Script Demonstration 2: Tabular Data Wrangling Part 2: Advanced Filtering, Multi-Segment GroupBy, and Financial Auditing

```
div_summary = df.groupby("Division_Name").agg(  
    Total_Budget_USD=("Budget_Expense_USD", "sum"),  
    Total_Actual_USD=("Actual_Expense_USD", "sum"),  
    Transaction_Count=("Transaction_ID", "count")  
)  
div_summary["Variance_USD"] = div_summary["Total_Actual_USD"] -  
div_summary["Total_Budget_USD"]  
div_summary["Variance_Pct"] = (div_summary["Variance_USD"] /  
div_summary["Total_Budget_USD"]) * 100.0  
print(div_summary[["Total_Actual_USD", "Variance_USD",  
"Variance_Pct"]])
```

SYNTAX

Readable Structure

Observe the syntax and indentation that control execution.

RUNTIME

Verified Execution

Run the example in the integrated terminal and inspect the output.

BUSINESS VALUE

Applied Automation

Use the pattern to automate a repeatable business decision.

Script Demonstration 3: Tabular Data Wrangling Part 2: Advanced Filtering, Multi-Segment GroupBy, and Financial Auditing

```
dept_info = pd.DataFrame({
    "Cost_Center_Code": ["CC-101", "CC-202", "CC-303", "CC-404"],
    "VP_Lead": ["Sarah Chen", "Marcus Vance", "Elena Rostova",
"David Kim"]
    })
merged_ledger = pd.merge(df, dept_info, on="Cost_Center_Code",
how="left")
print(merged_ledger[["Transaction_ID", "Division_Name", "VP_Lead",
"Actual_Expense_USD"]].head(3))
```

SYNTAX

Readable Structure

Observe the syntax and indentation that control execution.

RUNTIME

Verified Execution

Run the example in the integrated terminal and inspect the output.

BUSINESS VALUE

Applied Automation

Use the pattern to automate a repeatable business decision.

Exercise 1 and Exercise 2

EXERCISE 1: GUIDED

Foundational Practice

Follow the lab protocol, run the script, and verify its output with the session dataset.

EXERCISE 2: DEBUGGING

Applied Challenge

Extend the program, diagnose an intentional error, apply the verified correction, and confirm return code 0.